

Ranges and Tuples

CSCI 1101

2024-02-27

Iterating Over Lists of Indices

Sometimes we want to iterate over the elements of a list in a different order.

One way to do this is to iterate over one list containing *indices* for another list:

```
def rearrange(items , indices) :  
    ''' sig: (list a , list int) --> list a '''  
    acc = []  
    for index in indices :           # iterate over one list  
        acc.append(items[index])    # index into another  
    return acc
```

Solve the puzzle:

```
> letters = list('bdinow')  
> ''.join(rearrange(letters , ?what))  
'bowdoin'
```

Ranges

A **range** represents a sequence of numbers following some pattern.

The function `range : (int , int) --> iterator int` takes a **start number** and a **stop number** and returns the sequence of numbers *including* the start number but *excluding* the stop number (just like slices).

The result of this function is an iterator object that we can *listify* if needed:

```
list(range(1 , 11))
```

The start number is optional, if we omit it then 0 is assumed:

```
list(range(10))
```

Activity: Regular Polygons

Write a function `regular_polygon : (int , int) --> NoneType` that takes a number of sides and a side length and draws a *regular polygon* of the specified shape and size.



Hint: iterate over a range and don't forget to `import turtle`.

Tuples

A **homogeneous list** is a data structure for storing a *variable number* of elements, all of the *same type*.

We write lists surrounded by brackets with elements separated by commas:

```
["hello", "world", "!"] : list str
```

A **tuple** is a data structure for storing a *fixed number* of elements, each of a *specified type*.

We write tuples surrounded by parentheses with elements separated by commas:

```
("hello" , 42 , True) : tuple [str, int, bool]
```

homogeneous list types are parameterized by a *single element type*,

tuple types are parameterized by a *list of element types*.

Tuple Indexing and Slicing

Tuples are used to represent compound data of a given “shape”, corresponding to the number and types of their elements.

For example, we can represent a person by their last name, first name, and birth year using a tuple `[str, str, int]`:

```
john = ("Lennon" , "John" , 1940)
```

The positions in a tuple where data is stored are its **fields**.

We can extract a single field from a tuple using the index operator, `_[_]`:

```
year = john[2]
```

We can also project a sub-tuple from a tuple using the slice operator, `_[_:_]`:

```
name = john[0 : 2]
```

Tuple Assignment

Because the number of fields in a tuple is fixed, we can use **tuple assignment** to assign values to a tuple of variables at once:

```
(last , first) = name
```

This does the same thing as:

```
last = name[0]  
first = name[1]
```

Enumerating Lists

There is a function,

`enumerate : (list a) --> iterator (tuple [int, a])`
that packages each element of a list up with its index.

```
shopping = enumerate \  
    (["oat milk", "peanut butter", "grapes"])
```

This is useful if we want to know the positions of items in a list, for example to print only every other item:

```
def print_every_other(items) :  
    ''' sig: (list a) --> NoneType '''  
    for (index , item) in enumerate(items) :  
        if index % 2 == 0 :  
            print(str(item))
```


Activity: Getting Indices

Write a function `indices : (a , list a) --> list int` that returns the list of indices at which the given item occurs in the given list.

For example:

```
> indices("o" , list("bowdoin"))  
[1, 4]
```

Hint: use the accumulator pattern and `enumerate` the list.

Spreadsheets

A homogeneous list of tuples can act as a table or spreadsheet:

```
#signature:  list (tuple [str, str, int])
beatle_grades = \
[   #   last      ,   first      , grade
    ("Lennon"    , "John"      , 94) ,
    ("McCartney" , "Paul"     , 82) ,
    ("Harrison"  , "George"   , 79) ,
    ("Starr"     , "Ringo"    , 66)
]
```

We can use python to write spreadsheet functions:

```
def get_grade (record) :
    return record[2]
```

Activity: Averaging Grades

Write a function `average_grade` that takes a nonempty table of grades of type `list (tuple [str, str, int])` and returns the average grade as a `float`.

For example:

```
average_grade(beatle_grades)  
80.25
```

Hint: use the accumulator pattern and the `get_grade` function.

To Do

Finish *Homework 5: Lists and Loops* on CodeRunner by 11:00AM this Thursday.

Finish *Project 2 - What Does the Fox Say?* on CodeRunner by next Friday.